

# 逆ポーランド電卓

スタックを使うアルゴリズムの例として、基本情報処理技術者試験にもよく出題される「逆ポーランド電卓」を取り上げます。

**逆ポーランド記法 (Reverse Polish Notation, RPN)** と **スタック (Stack)** は、基本情報技術者試験でも非常によく組み合わせて出題される、コンピュータの数式処理の基本です。

私たちが普段使っている数式（中置記法）と何が違い、なぜコンピュータにとって都合が良いのかを分かりやすく解説します。

---

## 逆ポーランド記法（演算子が後ろにくる）

私たちが普段使う  $1 + 2$  のような式は、数と数の「中」に演算子があるため **中置記法** と呼ばれます。一方、逆ポーランド記法では、演算子を数の「後ろ（直後）」に書きます。

- **中置記法（普通の式）**：  $1 + 2$
- **逆ポーランド記法**：  $1\ 2\ +$

## 最大のメリット：カッコ ( ) が不要になる

例えば、次の2つの式を比べてみましょう。

- **式A**：  $1 + 2 * 3$ （掛け算が先なので、結果は7）
- **式B**：  $(1 + 2) * 3$ （カッコが先なので、結果は9）

中置記法では「掛け算を優先する」「カッコの中を先に計算する」という複雑なルールが必要ですが、逆ポーランド記法に変換すると、**カッコを使わずに計算順序を固定** できます。

- **式AのRPN**：  $1\ 2\ 3\ * \ +$ （2と3を掛け算し、その結果に1を足す）
- **式BのRPN**：  $1\ 2\ +\ 3\ *$ （1と2を足し算し、その結果に3を掛ける）

コンピュータは「左から右へ順番に読むだけ」で、優先順位を気にせず正しく計算できるようになります。

---

## スタックを使った計算の仕組み

逆ポーランド記法の式を計算するときに大活躍するのが、データ構造のスタック（最後に入れたものが最初に出る：LIFO）です。

ルールは非常にシンプルで、式を左から1つずつ読み進めながら、以下のように動かします。

1. **数字** が来たら：スタックにプッシュ（入れる）する。
2. **演算子 (+ や \* など)** が来たら：スタックから数字を **2つポップ（取り出す）** して計算し、その結果を再びスタックに **プッシュ（戻す）** する。

### $1\ 2\ +\ 3\ *$ の具体的なトレース：

- $1$  を読む → スタック：  $[1]$
- $2$  を読む → スタック：  $[1, 2]$

- **+** を読む → **2** と **1** を取り出して足算 ( $1 + 2 = 3$ ) → 結果を戻す。スタック: **[3]**
- **3** を読む → スタック: **[3, 3]**
- **\*** を読む → **3** と **3** を取り出して掛け算 ( $3 \times 3 = 9$ ) → 結果を戻す。スタック: **[9]**
- 最後にスタックに残った **9** が答えです。

#### 注意ポイント (引き算・割り算の順序):

スタックからデータを取り出す際、「**最初に取り出した方が右側 (後ろ)**」、「**2番目に取り出した方が左側 (前)**」になります。

例えば、**a b -** を計算するとき、最初に出るのが **b**、次が出るのが **a** なので、**a - b** を計算します。

## 逆ポーランド電卓プログラムの疑似言語

この「スタックを用いた逆ポーランド電卓」のアルゴリズムを、基本情報技術者試験の新形式疑似言語で記述したものです。

ここでは、式が **tokens[]** という配列に1つずつの要素 (文字型) として入っているものとします。(例: **["1", "2", "+"]**)

```

o整数型: rpnCalculator(文字型: tokens[], 整数型: n)
/* 整数を格納するスタックを用意 */
oスタック: stack
整数型: i, num1, num2, result

for (i を 0 から n - 1 まで 1 ずつ増やす)

    if (tokens[i] が数字のとき)
        /* 文字列を整数に変換してスタックに積む */
        stack.push(stringToInt(tokens[i]))

    else
        /* 演算子のとき: スタックから数値を2つ取り出す */
        num2 ← stack.pop() /* 最初に出たのが右の項 */
        num1 ← stack.pop() /* 2番目に出たのが左の項 */

        /* 演算子に応じて計算 */
        if (tokens[i] == "+")
            result ← num1 + num2
        elseif (tokens[i] == "-")
            result ← num1 - num2
        elseif (tokens[i] == "*")
            result ← num1 * num2
        elseif (tokens[i] == "/")
            result ← num1 / num2 /* 整数除算とする */
        endif

        /* 計算結果をスタックに戻す */
        stack.push(result)
    endif

```

```
endfor
```

```
/* 最後にスタックに残った1つの要素が最終的な計算結果 */  
return stack.pop()
```

---